

Reinforcement Learning

Peerapon S.

Machine Learning (1/67)

Topics

Examples and problem formulation

Policy and value function

Off-line learning

Q-learning

Temporal difference learning

ANOTHER VIEW OF MACHINE LEARNING

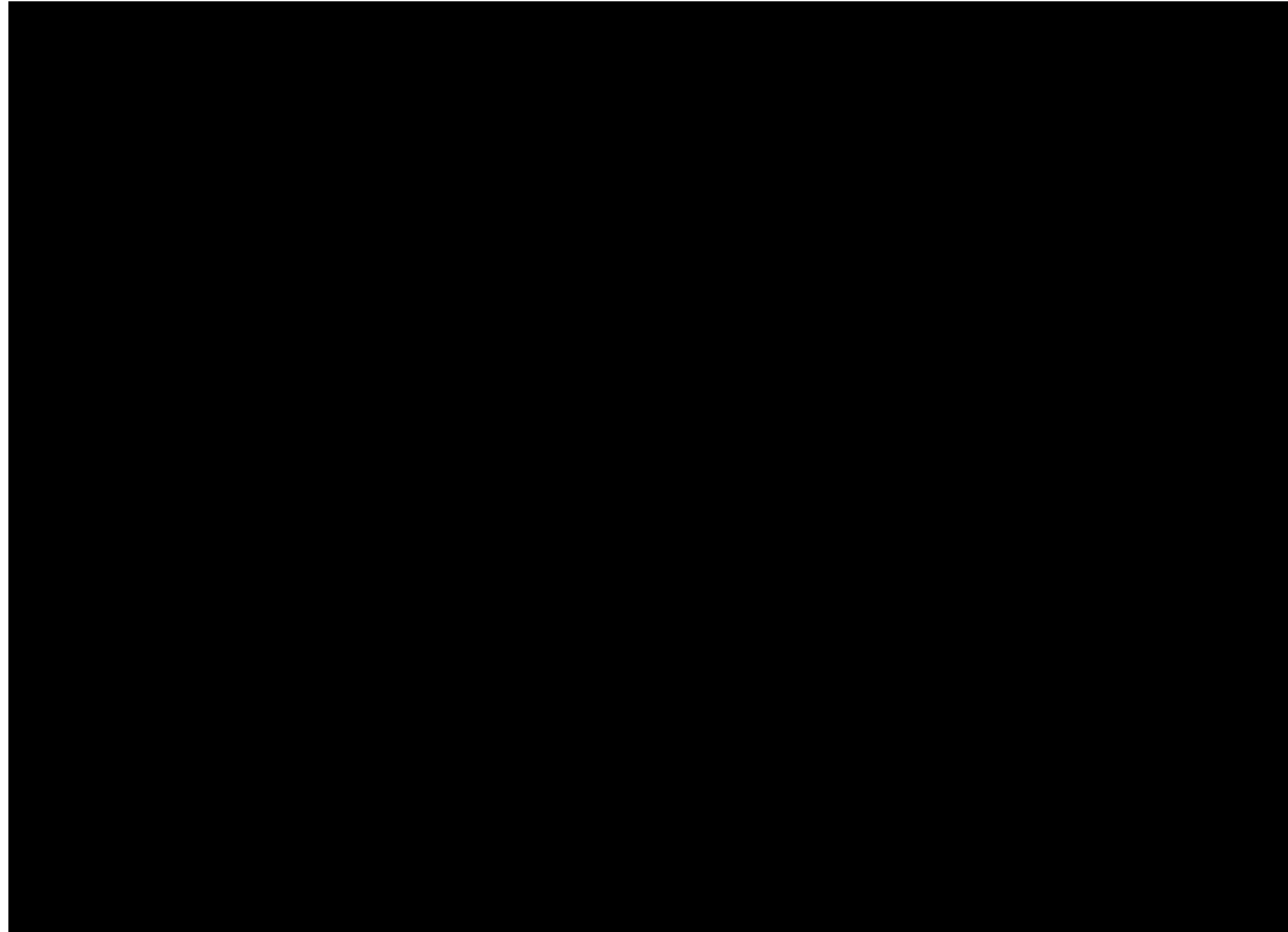


**TEACHING THE COMPUTER TO LEARN FROM
EXPERIENCES AND OPTIMIZE A GIVEN
PERFORMANCE INDEX AS THEY PRACTICE.**

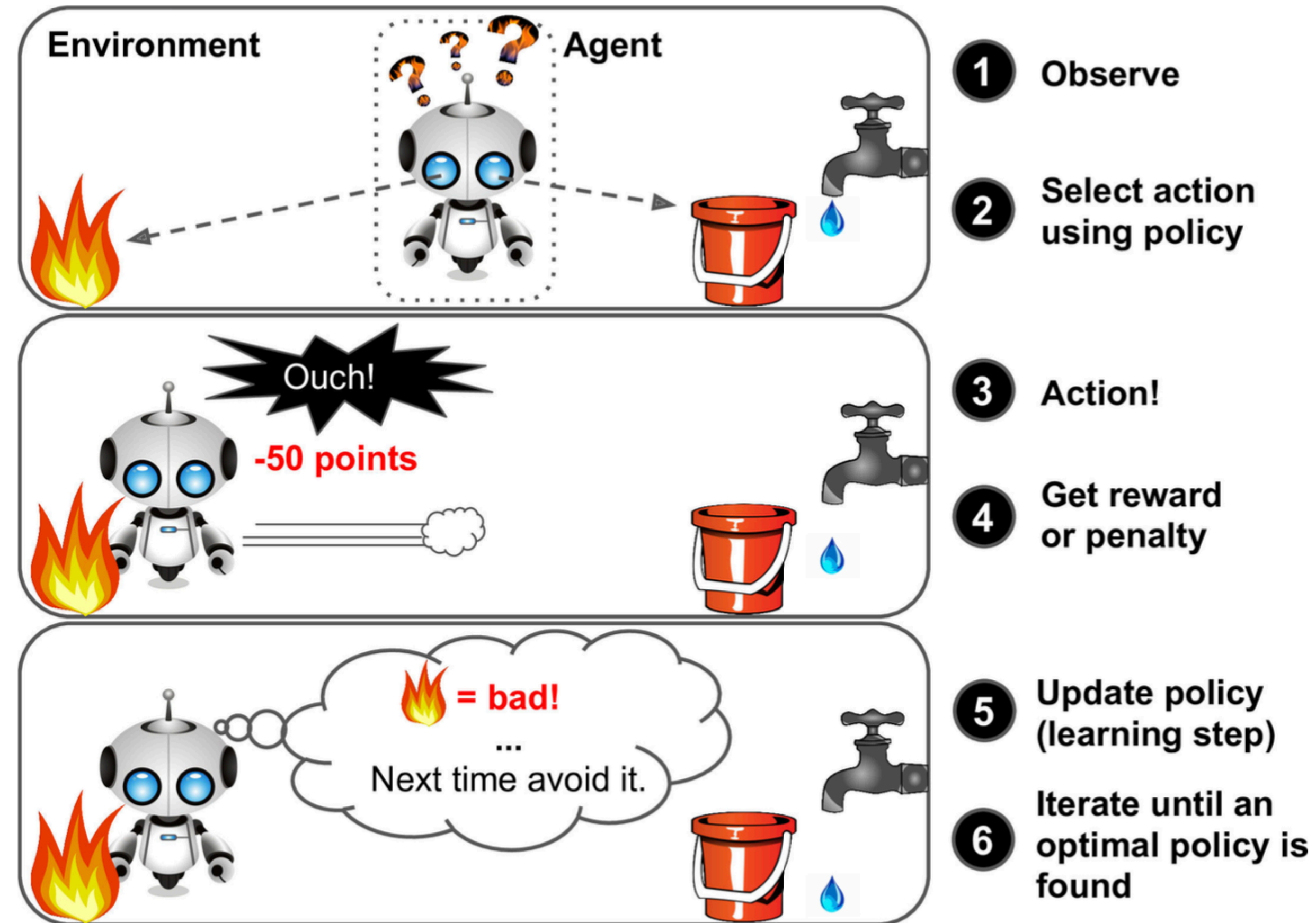
ATARI Game



Running Bot



Basic Concept



Problem Formulation

Agent (Decision maker) interacts with *environment* to complete a task.

At any point in time, the agent

- Observes the current *state* $s_t \in S$ of the environment.
- Selects an *action* $a_t \in A$ to perform, resulting in change of state to s_{t+1} .
- Receives an immediate *reward* (feedback) r_t from the environment.

define (S, A, R)

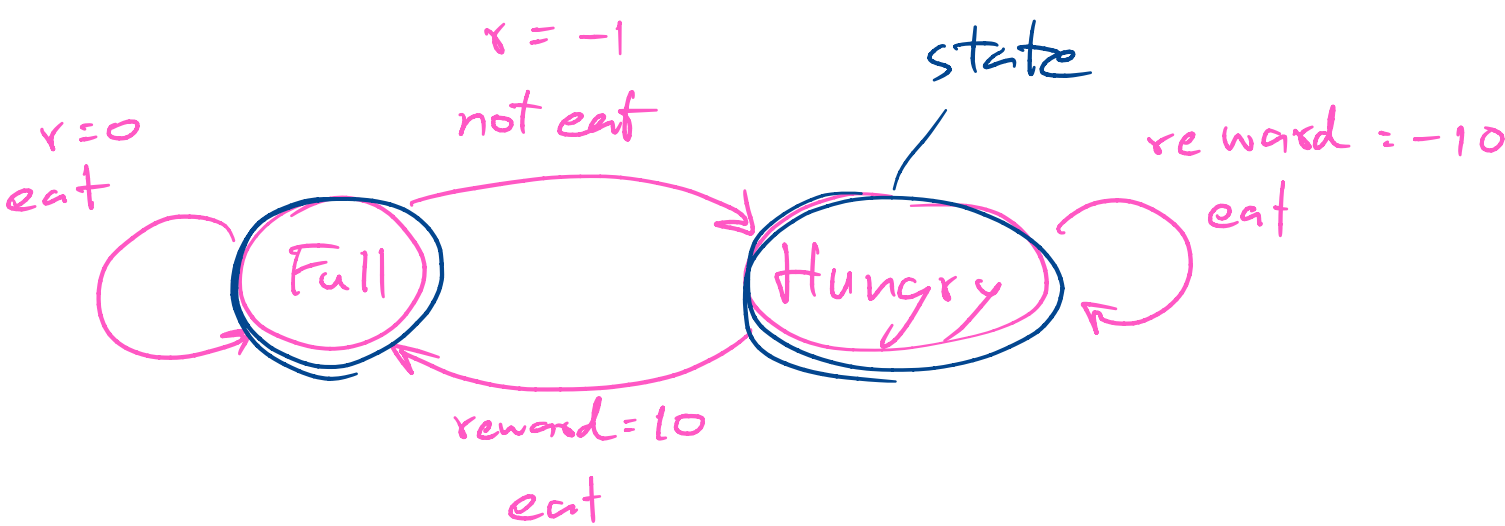
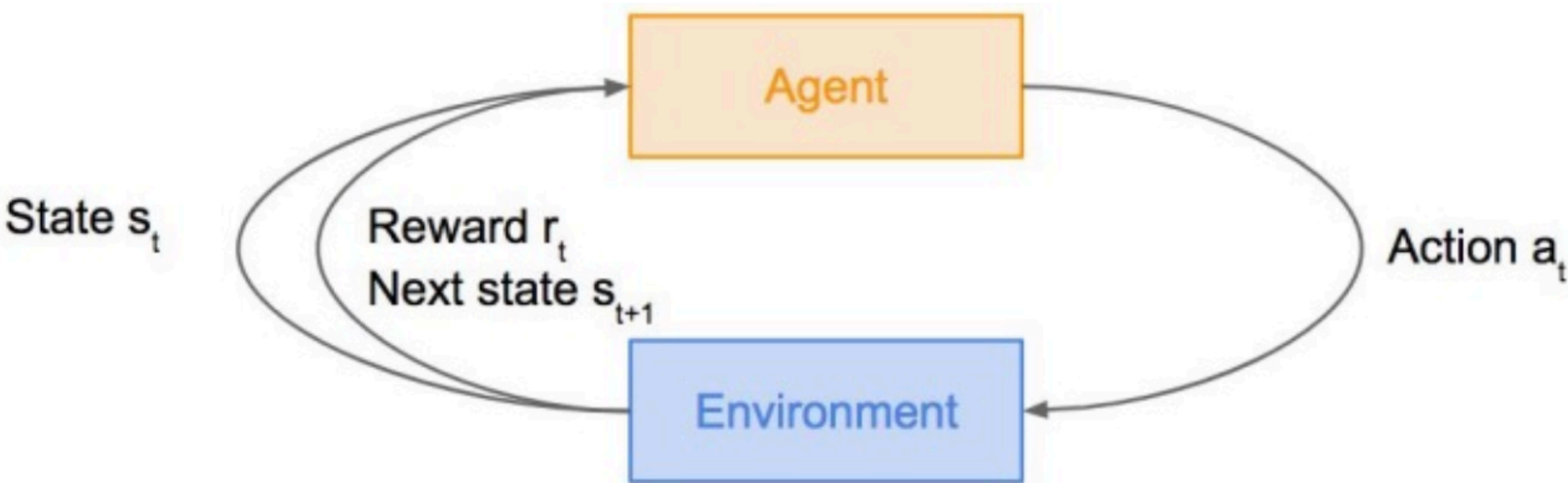
find $(s_0, a_0, r_0), (s_1, a_1, r_1), \dots, (s_n, a_n, r_n)$

s.t. $\sum r_i$ is maximized

Each attempt at the task is referred to as an *episode* or *trial*.

Goal: Complete the task as successfully as possible -- maximizing the *cumulative reward across an episode*.

$(s_1, a_1, r_1), (s_2, a_2, r_2), (s_3, a_3, r_3), \dots, (s_t, a_t, r_t), \dots$



reward

The *reward function* $r(\cdot)$ and *state transition function* $\delta(\cdot)$ are parts of the environment, referred to as *model of the world/environment*.



$$r(s,a)$$

state, action



state transition function

$$\delta(s,a) = s'$$

state s action a

state s'

Model possibilities:

- ◆ Known/Not known to the agent
- ◆ Independent or dependent on past states and actions
- ◆ Deterministic or Non-deterministic (Stochastic)

Ex: Atari's Arcade Game

State = Positions of remaining raw pixels

Action = Game controls - Left, Right

Reward = Score increase/decrease at each time step

Task = Complete the game with least number of actions.



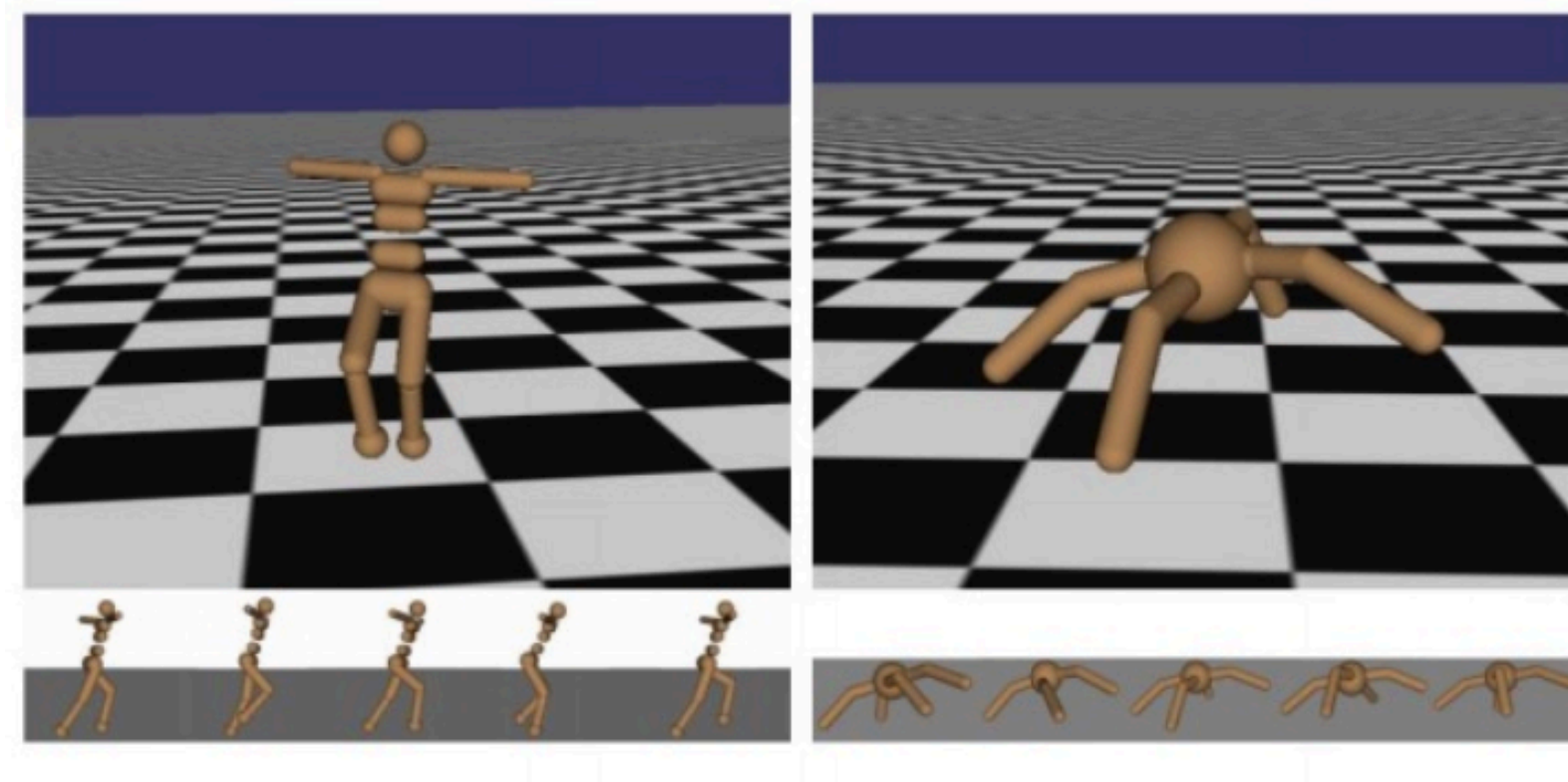
Ex: Robot Locomotion

State = Leg/body positions

Action = Leg/body movement

Reward = +1 if not falling down, 0 otherwise

Task = Make the robot move forward without falling down to the destination.

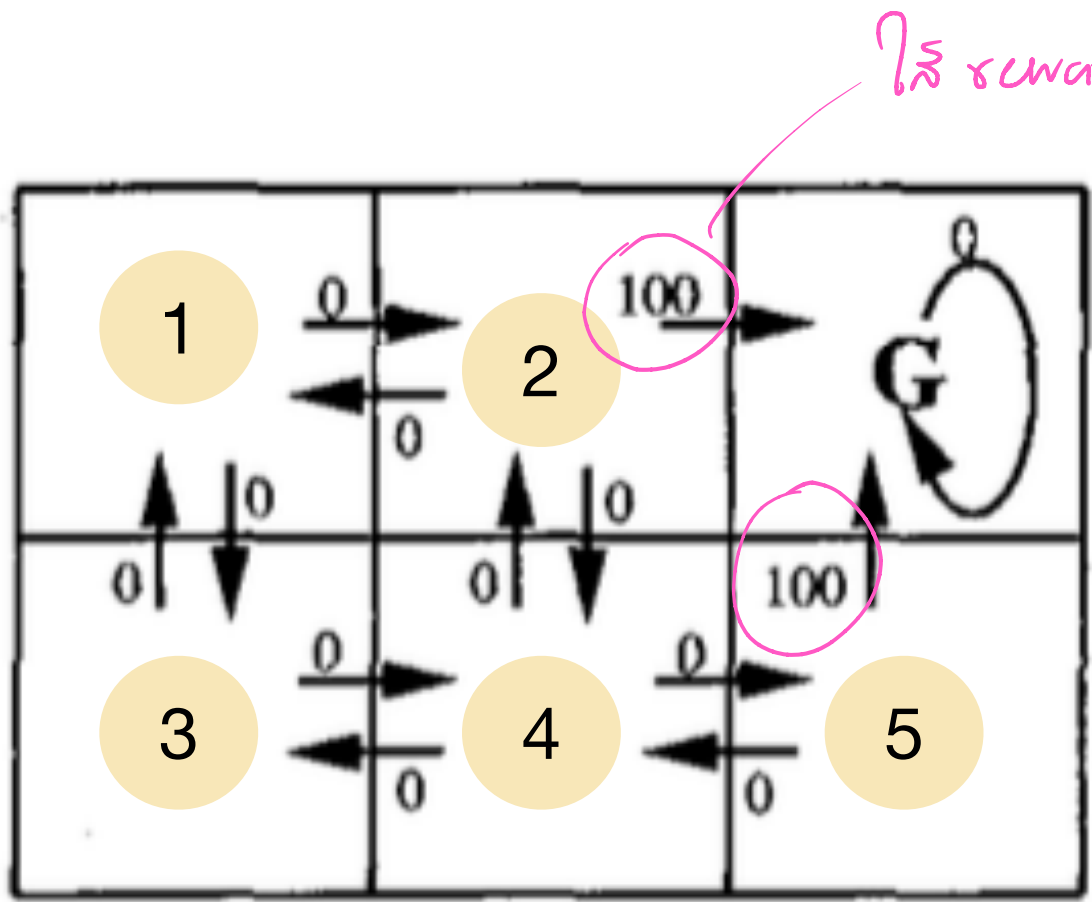


Ex: Six-grid-squares Environment

Assume $r(\cdot)$ and $\delta(\cdot)$ being deterministic and independent from past states and actions so that $r_t = r(s_t, a_t)$ and $s_{t+1} = \delta(s_t, a_t)$.

- What is the set of states ?
- What is the set of actions ?
- What is the reward ?

Task = Go the cell G (Goal/Terminal/Absorbing state) with least number of moves from any initial cell.



Immediate reward values $r(a, s)$

$S = \{ 1, 2, 3, 4, 5, G \}$
 $A = \{ (1 \rightarrow 2), (1 \rightarrow 3), \dots \}$ หรือ $\{ W, A, S, D \}$

at state $s_t \rightarrow$ do action a_t

action: action

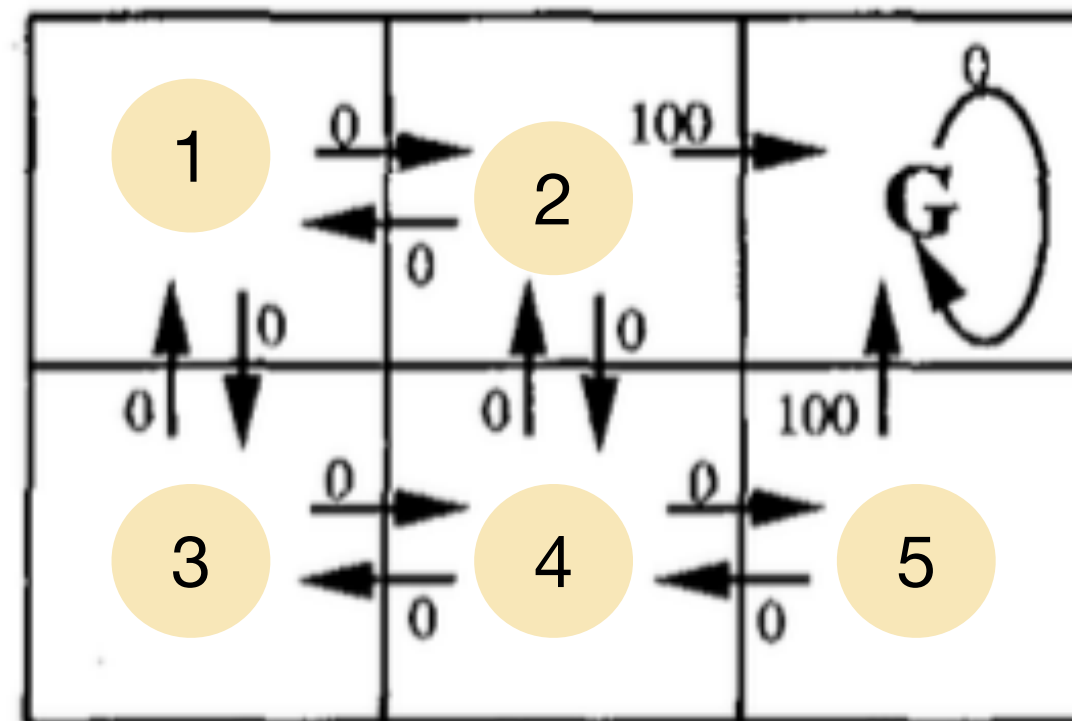
Policy

Policy = at state $s_t \rightarrow$ do action a_t

A policy (function) $\pi : S \rightarrow A$ is a **mapping** from $s_t \in S$ to $a_t \in A$, i.e., what action $a_t = \pi(s_t)$ to take in each state s_t .

Given $\pi(s)$, the action in state s is completely determined by s .

policy have
1 input (s)



design policy

Ex: policy state 1 $\rightarrow$ 2 50%
3 50%

Ex: **Greedy action selection policy**

Always take the action yielding the highest immediate reward.

Fail to handle different characteristics of rewards, e.g., delayed or contradictory.

reward delay ใน action

state ① ได้ reward ② ไม่ได้ reward

state ② $\rightarrow$ ③ ได้ reward

local minimum

Need a different measure to decide an action in a given state.

164 164 1 → 2 → 4 → 5 → 6
 96 reward 200000000

Goal: Maximize $V^\pi(s_t)$

Value Function -- Discounted Reward



Defined as a cumulative reward ← remember? this is what we need to maximize expected to earn when the agent is in state s_t and follows the policy π .

For $0 \leq \gamma < 1$, the *discounted (cumulative) reward* obtained from using the policy π is defined by

value (reward) → $V^\pi(s_t) \triangleq r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$

γ param ← this is to tune discount factor
 policy
 value: state & value
 1 input
 reward: r_t
 reward: r_{t+1}
 equal by definition sign

ที่ 8 จากข้อ 1, 2, 3, ... ตามพจน์ที่ γ ... เราต้องให้ค่าที่สอดคล้องกับ
 ปัจจุบันที่รู้
 2018.11.16

$V^\pi(s)$ has the recurrence relation with that of next state s' . $\rightarrow V^\pi(s_t) = r_t + \gamma V^\pi(s_{t+1})$

What is the implication of discount factor γ ?

↳ higher discount $\rightarrow$ higher consideration of future states

Optimal Policy

The goal of RL is the agent learns the optimal policy -- Maximizing the cumulative reward in the long run.

The optimal policy is defined as $\pi^*(s) \triangleq \arg \max_{\pi} V^{\pi}(s), \forall s$

with the optimal policy value $V^{\pi^*}(s) \triangleq V^*(s)$

policy ကံကံနဲ့ *စံကံနဲ့*

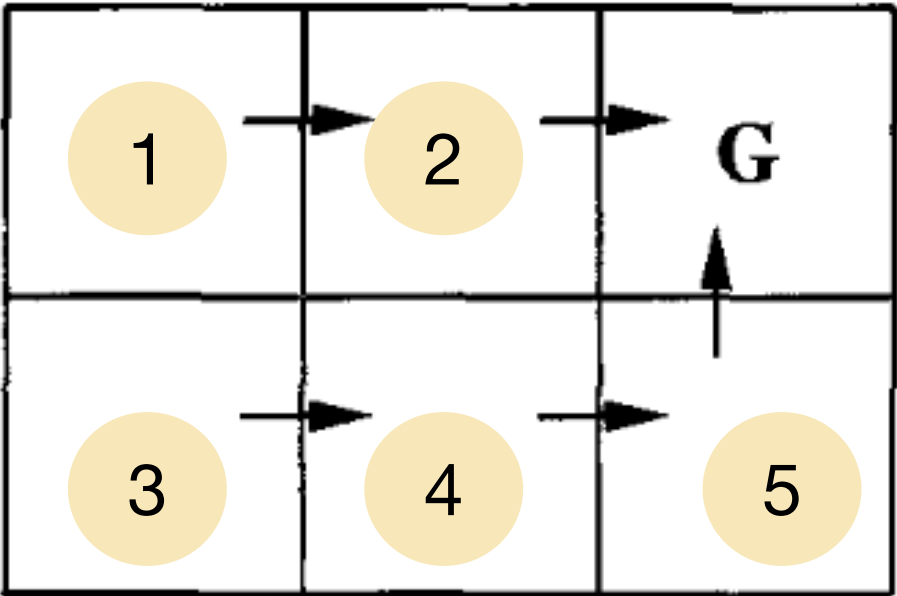
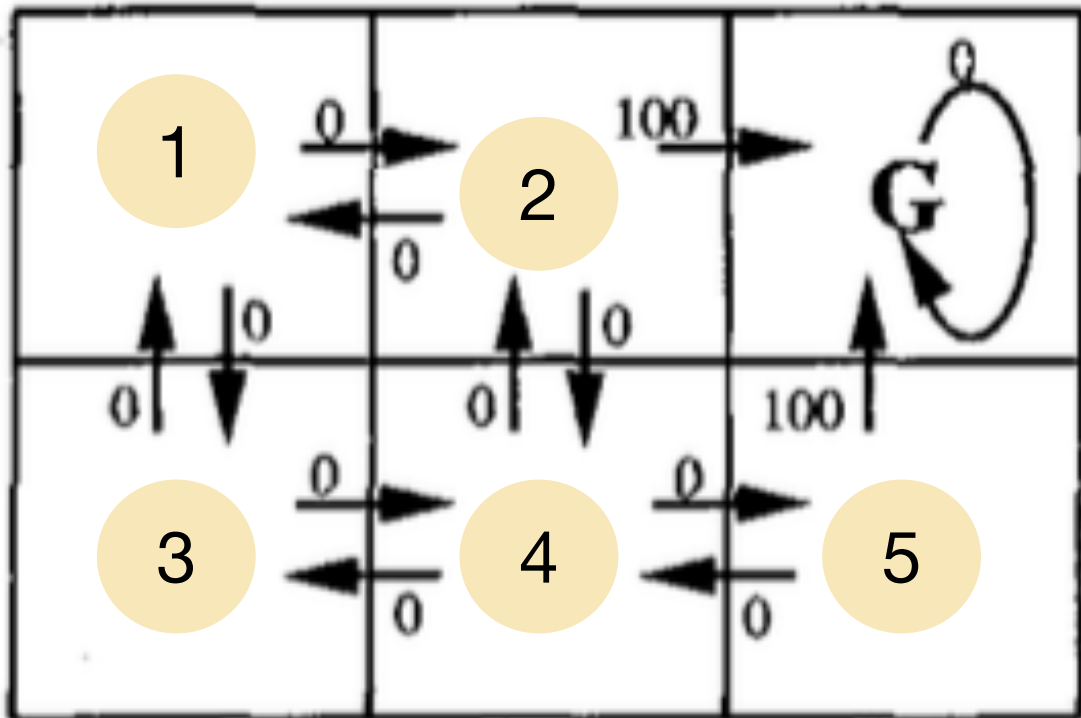
just find $\pi = \pi^$ that maximize V for all $s \in S$,
that's the optimal policy*

which gives optimal reward/value

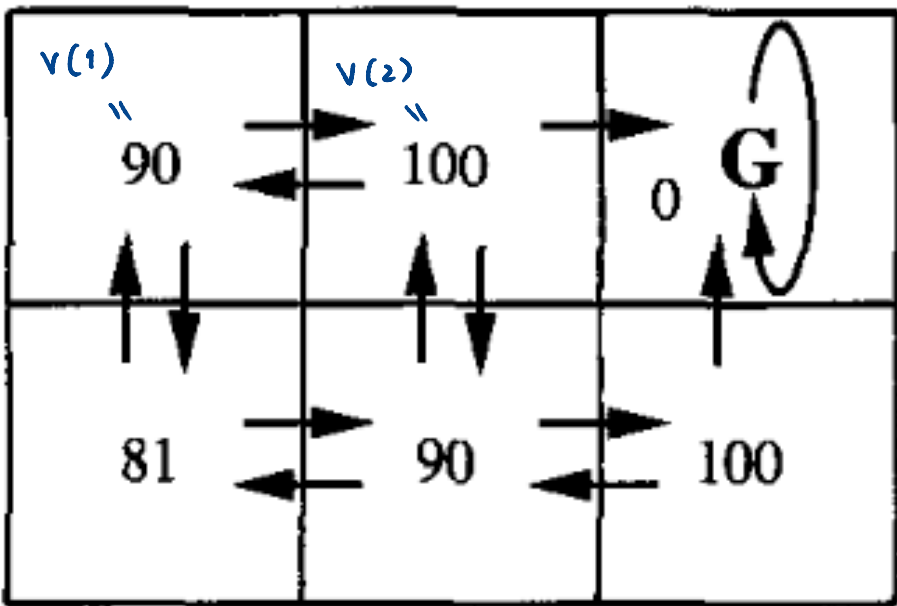
any state ကဲ့သို့ value နဲ့နဲ့

So, the optimal policy always yields the highest policy value for any starting state.

Example of Optimal Policy



One optimal policy



$V^*(s)$ values

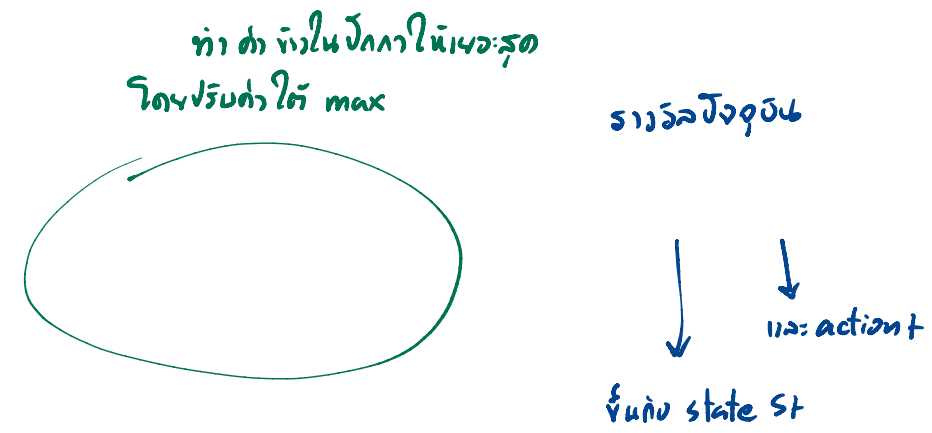
Ex:

$$\max_y (-y^2) = 0$$

$$\max_y (-y^2 + 4) = 4$$

$$\operatorname{argmax}_y (-y^2 + 4) = 0$$

(ค่า $(-y^2+4)$ สูงสุด ที่หาค่าได้)
 (เมื่อค่า y ที่หาค่าได้)
 $(-y^2+4)$ ค่าสูงสุด



นั่นคือว่า greedy มองแค่
 $\therefore$ greedy ขาดมุมมอง

ค่า v ที่ให้รู้ เกิดจาก reward ปัจจุบัน + reward อนาคต

reward ปัจจุบัน

next state

~~~~~

~~~~~  
 reward อนาคต

Bellman's Equation
 (Bellman's Optimality
 Condition)

ปรับ $V(r)$ เรือง จนกว่าดำจ:นี้



policy!



↑
state
↓
action
↓
state

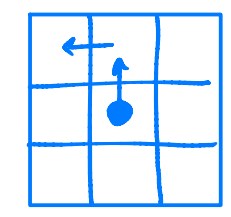
policy
random



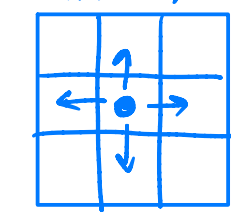
on-policy

policy

Q learning



V



One must express $Q(s_t, a_t)$ in the recurrence relation form.

The agent can iteratively approximate the Q function based on *observed rewards and next states* of each action taken without the knowledge of $r(s, a)$ and $\delta(s, a)$.

$$Q(s_t, a_t) = \max_{a_{t+1}, a_{t+2}, \dots} \left\{ r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \gamma^2 r(s_{t+2}, a_{t+2}) + \dots \right\}$$
$$Q(s, a) = r(s, a) + \gamma \max_{a'} Q(\delta(s, a), a') \quad (6)$$

$$\text{ကျွန်ုပ် } V = r + \gamma V(s)$$

Q-Learning Algorithm

From (6), by observing reward $r = r(s, a)$ and next state $s' = \delta(s, a)$, the learner can iteratively approximate $Q(s, a)$ with:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a') \quad (7)$$
$$s \leftarrow s'$$

The optimal policy is determined from (5a).

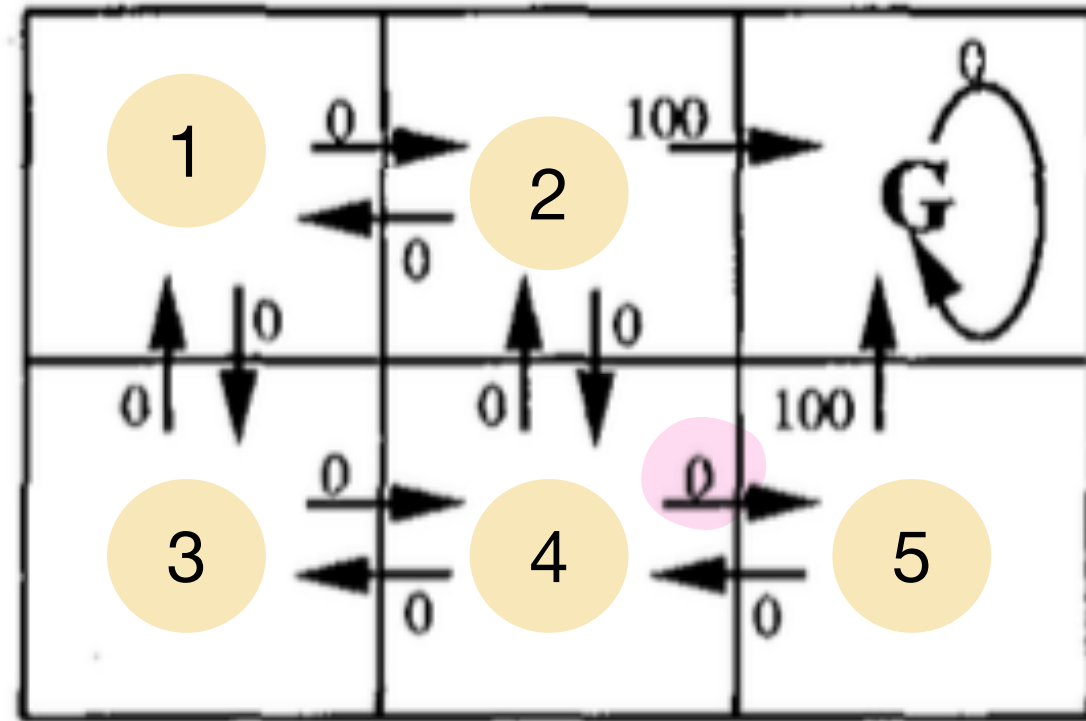
Algorithm (Q-Learning) :

First Loop :

1. Choose a random initial state
2. Choose a random action
3. Determine after taking action
what are the next state & reward
4. Update Q table of state/action
done in this loop

Latter Loops :

1. Choose a random initial state
2. Choose a best action determined so far ← or do random sometimes
to increase exploration
3. Determine after taking action
what are the next state & reward
4. Update Q table of state/action
done in this loop



Q-table at the start of episode 1

	Left	Right	Up	Down
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0
5	0	0	0	0
G	0	0	0	0

เริ่มไปเรื่อย ๆ ในแต่ละรอบ update
จนครบพอใจ

Episode 1 (Initial state 4)

ตอนแรก
เริ่มที่ state 4
ตอนแรกมี reward 0
100 state 5
t=0, s = 4, a = right, s' = 5

	Left	Right	Up	Down
1				
2				
3				
4		0		
5				
G				

เริ่ม
t=1, s = 5, a = up, s' = G

	Left	Right	Up	Down
1				
2				
3				
4		0		
5			100	
G				

$$\hat{Q}(4, \text{right}) \leftarrow r(4, \text{right}) + \gamma \max_{a'} [\hat{Q}(s, a')]$$

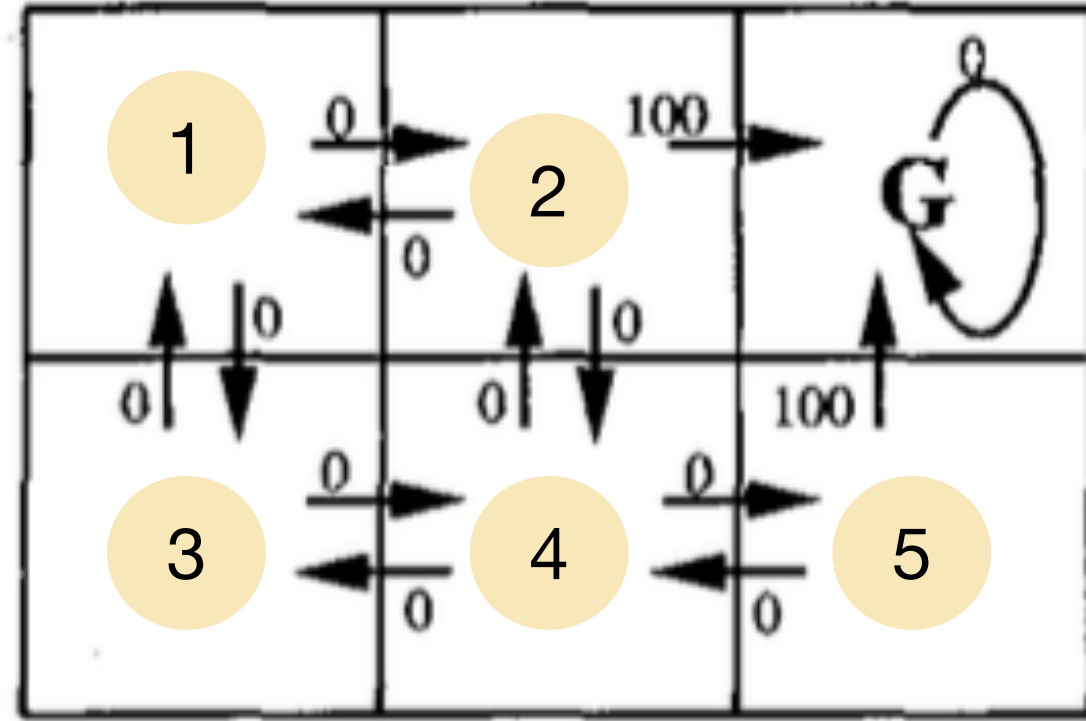
$$\leftarrow 0 + 0.8(0)$$

$$= 0$$

$$\hat{Q}(5, \text{up}) \leftarrow r(5, \text{up}) + \gamma \max_{a'} [\hat{Q}(G, a')]$$

$$\leftarrow 100 + 0.8(0)$$

Q-table at the start of episode 2



	Left	Right	Up	Down
1				
2				
3				
4				
5			100	
G				

blank = 0

Episode 2 (Initial state 4)

t=0: s = 4, a = right, s' = 5

	Left	Right	Up	Down
1				
2				
3				
4		80		
5			100	
G				

t=1: s = 5, a = up, s' = G

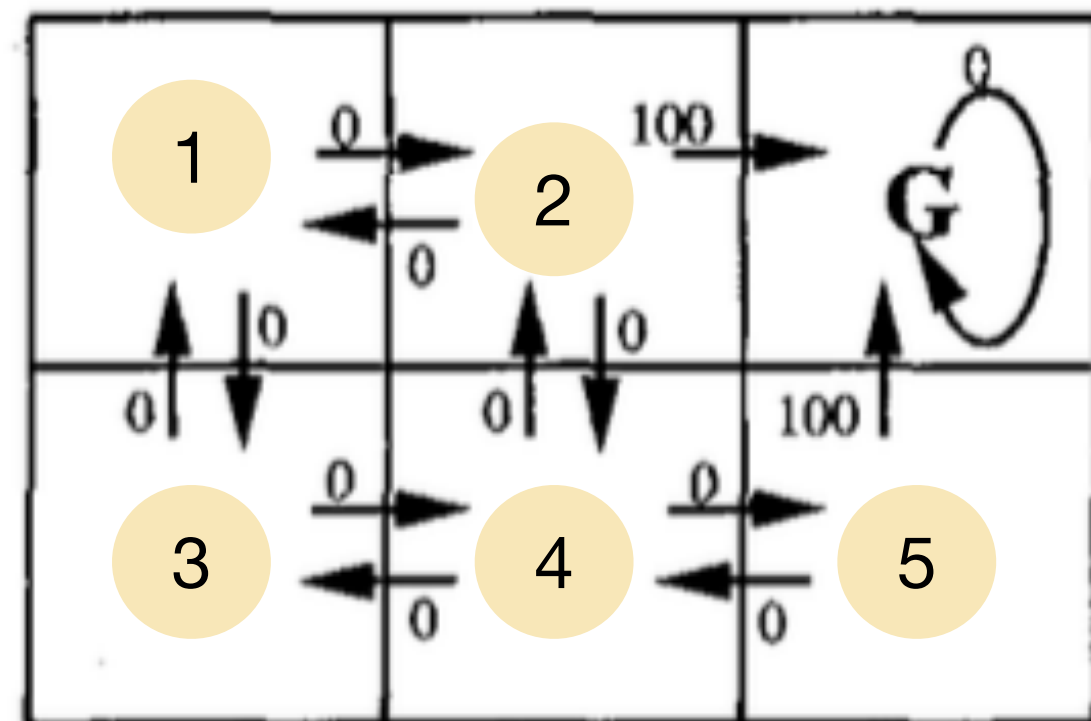
	Left	Right	Up	Down
1				
2				
3				
4		80		
5			100	
G				

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a'} [\hat{Q}(s, a')]$$

$$\leftarrow 0 + 0.8(100) = 80$$

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a'} [\hat{Q}(G, a')]$$

$$\leftarrow 100 + 0.8(0)$$



Q-table at the start of episode 3

	Left	Right	Up	Down
1				
2				
3				
4		80		
5			100	
G				

Episode 3 (Initial state 2)

t=0: s = 2, a = down, s' = 4

	Left	Right	Up	Down
1				
2				64
3				
4		80		
5			100	
G				

$$\hat{Q}(2, \text{down}) \leftarrow r(2, \text{down}) + \gamma \max_{a'} [\hat{Q}(4, a')] \leftarrow 0 + 0.8(80) = 64$$

optimal $a' = \text{right}$

t=1: s = 4, a = right, s' = 5

	Left	Right	Up	Down
1				
2				
3				
4				
5				
G				

no update

t=2: s = 5, a = up, s' = G

	Left	Right	Up	Down
1				
2				
3				
4				
5				
G				

no update

สุมเงิน ?

ทำตามทฤษฎีที่รู้ดีที่สุด เริ่ม เก็บเงิน 1 ถึงไป 2 

s	1	2	3	4	5	G
$V^*(s)$	80	100	64	80	100	0
$\pi^*(s)$	2	G	1, 4 ↖ ↗	2, 5 ↑	G	G

ลบออกจากทฤษฎีเราตาม บัญชี :: อาจเกิด local optima อยู่

Examples of Behavior Policy

Greedy action selection policy

Random action selection policy

ϵ -greedy selection policy *สมการ Explore ส่วนหนึ่ง แล้วด้วย exploit*

Gibbs or Boltzmann action selection policy

คำนวณค่าเลือก: ให้มันเลือกตามค่า


$$\mathbb{P}\{a_i \mid s\} = \frac{e^{\hat{Q}(s,a_i)/\tau}}{\sum_j e^{\hat{Q}(s,a_j)/\tau}}, \quad \tau > 0$$

↑ This is like "softmax"

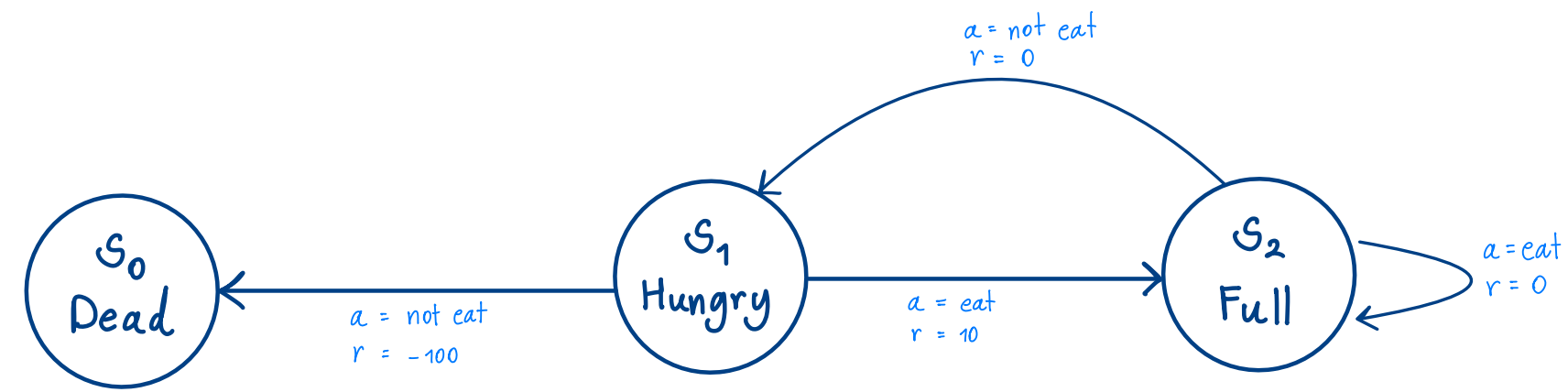
*Calculate prob. of taking an action
based on current Q-value of the action*

ນາວທີ່ ຢູ່ state ເດີມ ກໍ່ action ເດີມ ແຕ່ພວກລູກໄດ້ເຫັນວ່າເດີມ

157 model ວັນຄັ້ງ Markov



Deterministic

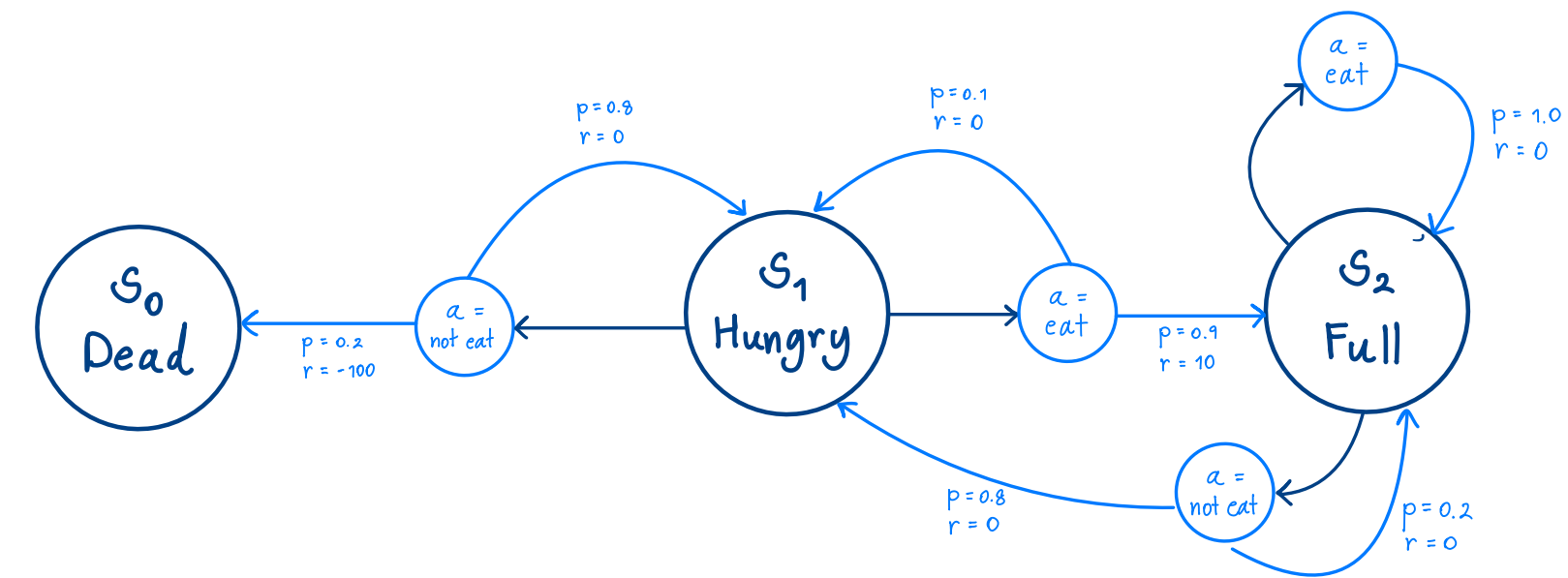


We can just write state change as function

$$S(\text{Hungry}, \text{eat}) = \text{Full}$$

$$S(\text{Full}, \text{not eat}) = \text{Hungry}$$

Non-Deterministic

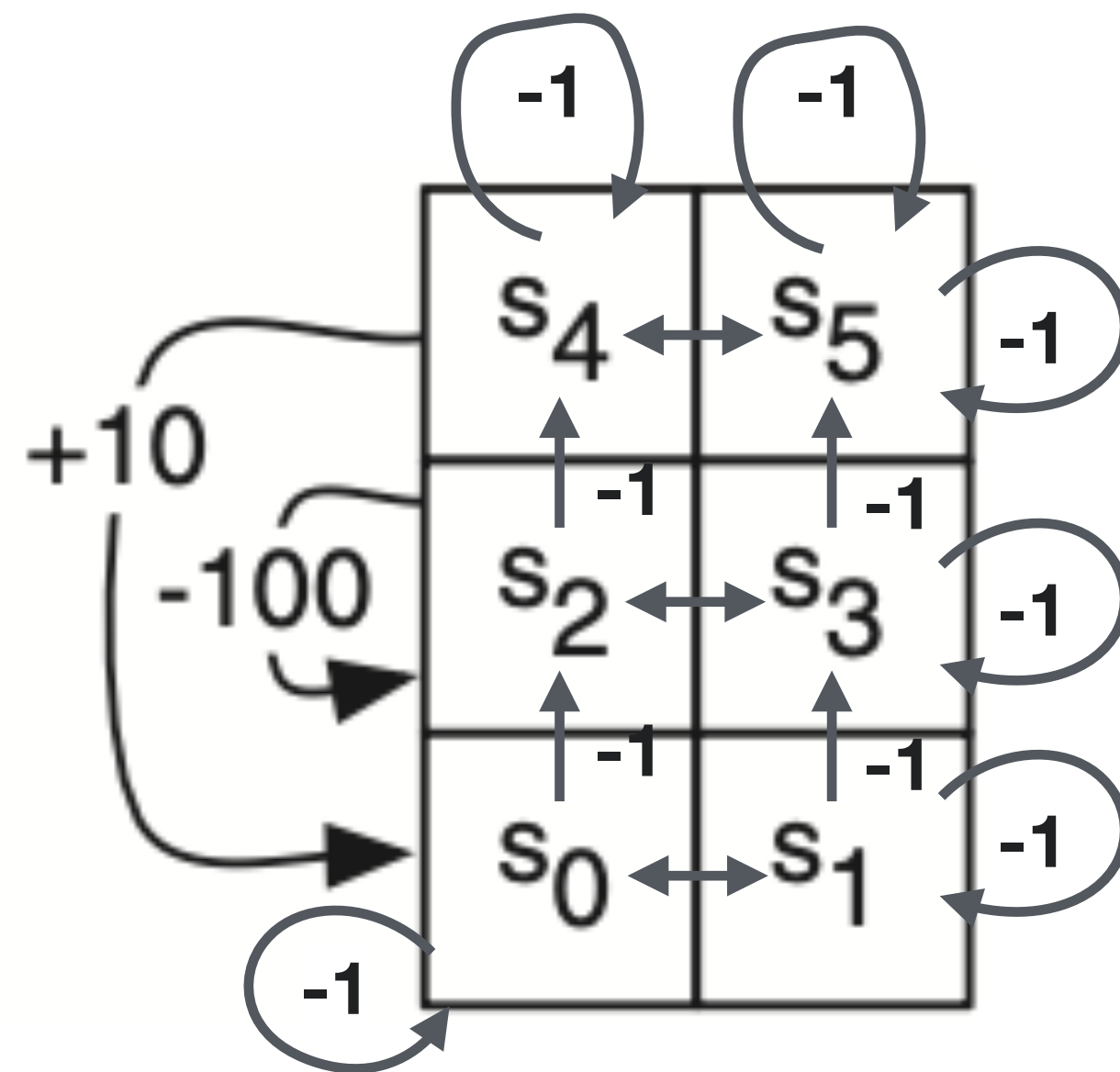


We have to use prob. to model state change

$$P(\text{Full} \mid \text{Hungry}, \text{eat}) = 0.9$$

$$P(\text{Hungry} \mid \text{Hungry}, \text{eat}) = 0.1$$

Example



Set of actions and rewards

upC (for “up carefully”) The agent goes up, except in states s_4 and s_5 , where the agent stays still, and has a reward of -1 .

right The agent moves to the right in states s_0, s_2, s_4 with a reward of 0 and stays still in the other states, with a reward of -1 .

left The agent moves one state to the left in states s_1, s_3, s_5 . In state s_0 , it stays in state s_0 and has a reward of -1 . In state s_2 , it has a reward of -100 and stays in state s_2 . In state s_4 , it gets a reward of 10 and moves to state s_0 .

up With a probability of 0.8 it acts like *upC*, except the reward is 0. With probability 0.1 it acts as a *left*, and with probability 0.1 it acts as *right*.

Determine the policy to maximize the long-term cumulative reward.

Markov Chain

$$P(s_0 | s_4, \text{Up}) = 0.1$$

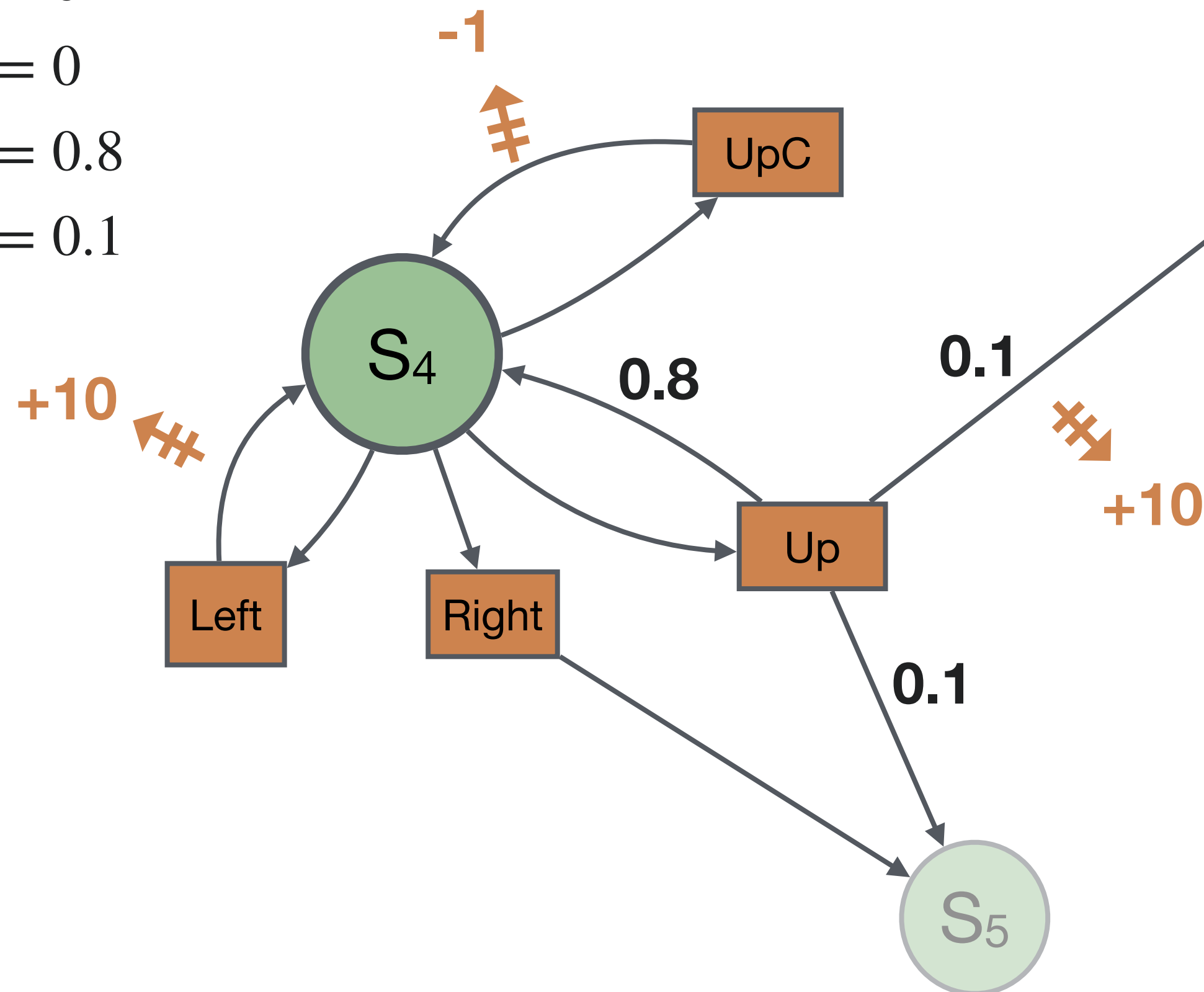
$$P(s_1 | s_4, \text{Up}) = 0$$

$$P(s_2 | s_4, \text{Up}) = 0$$

$$P(s_3 | s_4, \text{Up}) = 0$$

$$P(s_4 | s_4, \text{Up}) = 0.8$$

$$P(s_5 | s_4, \text{Up}) = 0.1$$



2701526 diagram

upC (for “up carefully”) The agent goes up, except in states s_4 and s_5 , where the agent stays still, and has a reward of -1 .

right The agent moves to the right in states s_0, s_2, s_4 with a reward of 0 and stays still in the other states, with a reward of -1 .

left The agent moves one state to the left in states s_1, s_3, s_5 . In state s_0 , it stays in state s_0 and has a reward of -1 . In state s_2 , it has a reward of -100 and stays in state s_2 . In state s_4 , it gets a reward of 10 and moves to state s_0 .

up With a probability of 0.8 it acts like *upC*, except the reward is 0 . With probability 0.1 it acts as a *left*, and with probability 0.1 it acts as *right*.

Ex: ^{ค่าเฉลี่ย} sequence of observed values $v_1, v_2, v_3, \dots$ → คือ data "ค่า: ค่า นารด้วย และ, หัสนมด
 หัสนมด $\pm$ ค่า → คือ data ใน หัสนมด
 TD คือการหาค่าเฉลี่ยของ data ใน หัสนมด data หัสนมด

Temporal Difference (TD) Error

Consider sequence of observed numeric values $v_1, v_2, v_3, \dots$

Let A_k be an estimate of the expected value of (most recent) k data points. Then,

ex. $v = 3, 1, 2, 4, 5 \rightarrow \mu = 3$

$$A_k = \frac{v_1 + v_2 + \dots + v_k}{k}$$

$$= \overset{\text{mean}}{A_{k-1}} + \alpha_k \underbrace{(v_k - A_{k-1})}_{\text{data ใน mean}}$$

การหาค่าเฉลี่ย
ใน หัสนมด $\alpha_k = \frac{1}{k}$

$$\alpha_k = \left(\frac{1}{k} \right)$$

$$A_1 = 3$$

$$A_2 = 3 + \frac{1}{2}(1-3) = 2$$

$$A_3 = 2 + \frac{1}{3}(2-2) = 2$$

$$A_4 = 2 + \frac{1}{4}(4-2) = 2.5$$

$$A_5 = 2.5 + \frac{1}{5}(5-2.5) = 3 = \mu$$

Temporal difference (TD) error

- ◆ How does A_k change with the latest observed value?
- ◆ How much A_k change with the latest observed value?

In RL, more recent values are more accurate and could be weighted more as the learning progress.

Q-Learning in a Non-Deterministic Environment

One form of *temporal difference learning* where the TD error is the change in Q value.

The agent learns from the history of interaction as a sequence of *experiences* $\langle s, a, r, s' \rangle$ using

$$\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha [\underbrace{r(s, a) + \gamma \max_{a'} \hat{Q}(s', a')}_{\text{Actual return = Observed reward + Discounted future reward}} - \underbrace{\hat{Q}(s, a)}_{\text{Current estimated return}}], \quad 0 < \alpha < 1$$

Handwritten notes in the image:
- Above $\hat{Q}(s, a)$: $\hat{Q}$ value
- Above $\hat{Q}(s, a)$: $\hat{Q}$ value
- Above α : α learning rate
- Above $r(s, a)$: r reward
- Above γ : γ discount factor
- Above $\max_{a'} \hat{Q}(s', a')$: $\max$ over next state actions
- Above $\hat{Q}(s', a')$: $\hat{Q}$ value
- Above $\hat{Q}(s, a)$: $\hat{Q}$ value
- Above the subtraction sign: $+$ TD error

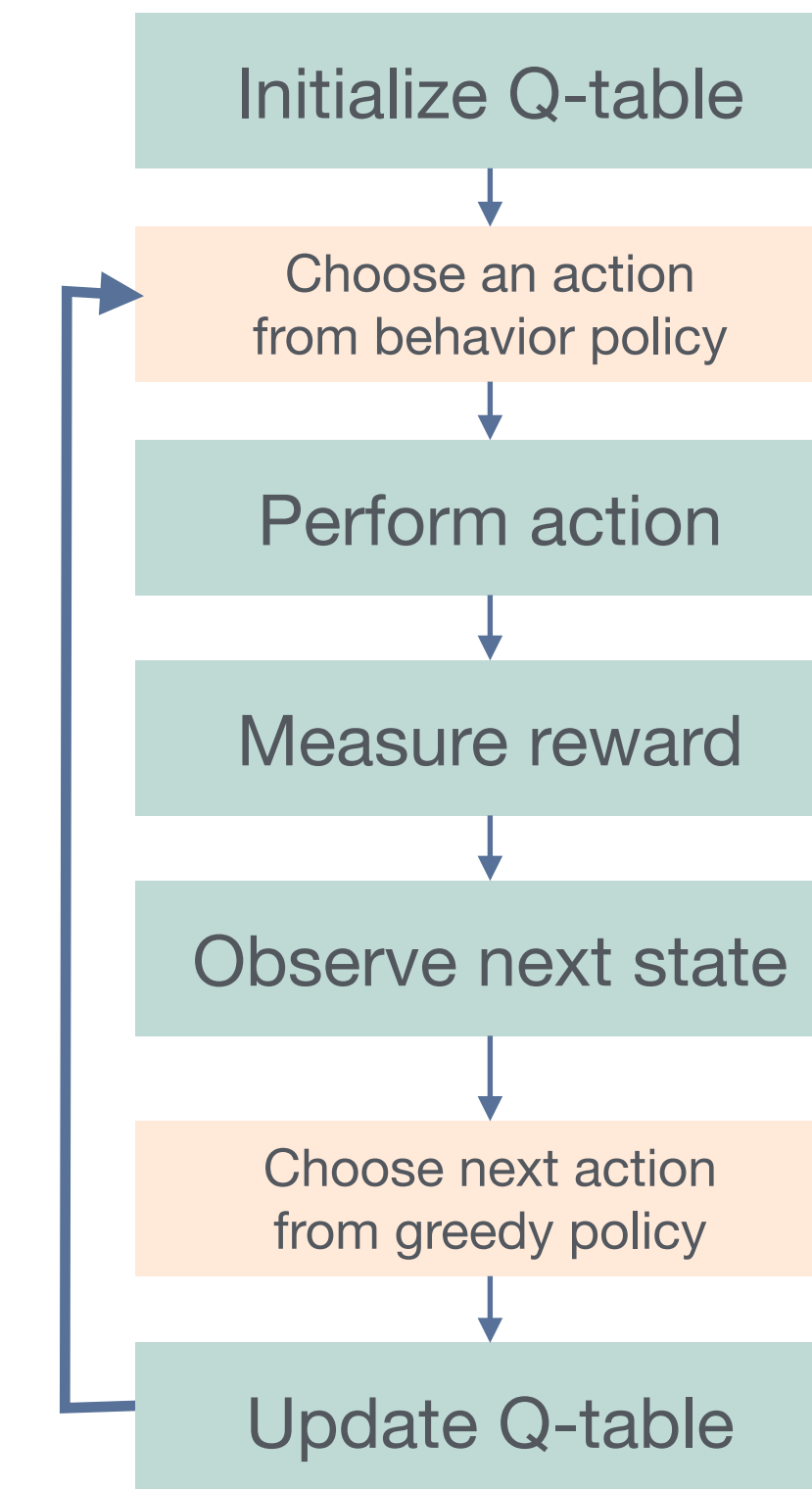
- ◆ Current action a in state s chosen by using the behavior policy.
- ◆ Next state action a' chosen to maximize the actual return (the greedy policy).

Regarded as an *off-policy* learning algorithm - Learn an optimal policy regardless of the behavior policy used during learning provided that it explores enough state-action pairs.

```

1: controller  $Q\text{-learning}(S, A, \gamma, \alpha)$ 
2:   Inputs
3:      $S$  is a set of states
4:      $A$  is a set of actions
5:      $\gamma$  the discount
6:      $\alpha$  is the step size
7:   Local
8:     real array  $Q[S, A]$ 
9:     previous state  $s$ 
10:    previous action  $a$ 
11:    initialize  $Q[S, A]$  arbitrarily
12:    observe current state  $s$ 
13:    repeat
14:      select and carry out an action  $a$ 
15:      observe reward  $r$  and state  $s'$ 
16:       $Q[s, a] \leftarrow Q[s, a] + \alpha (r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$ 
17:       $s \leftarrow s'$ 
18:    until termination

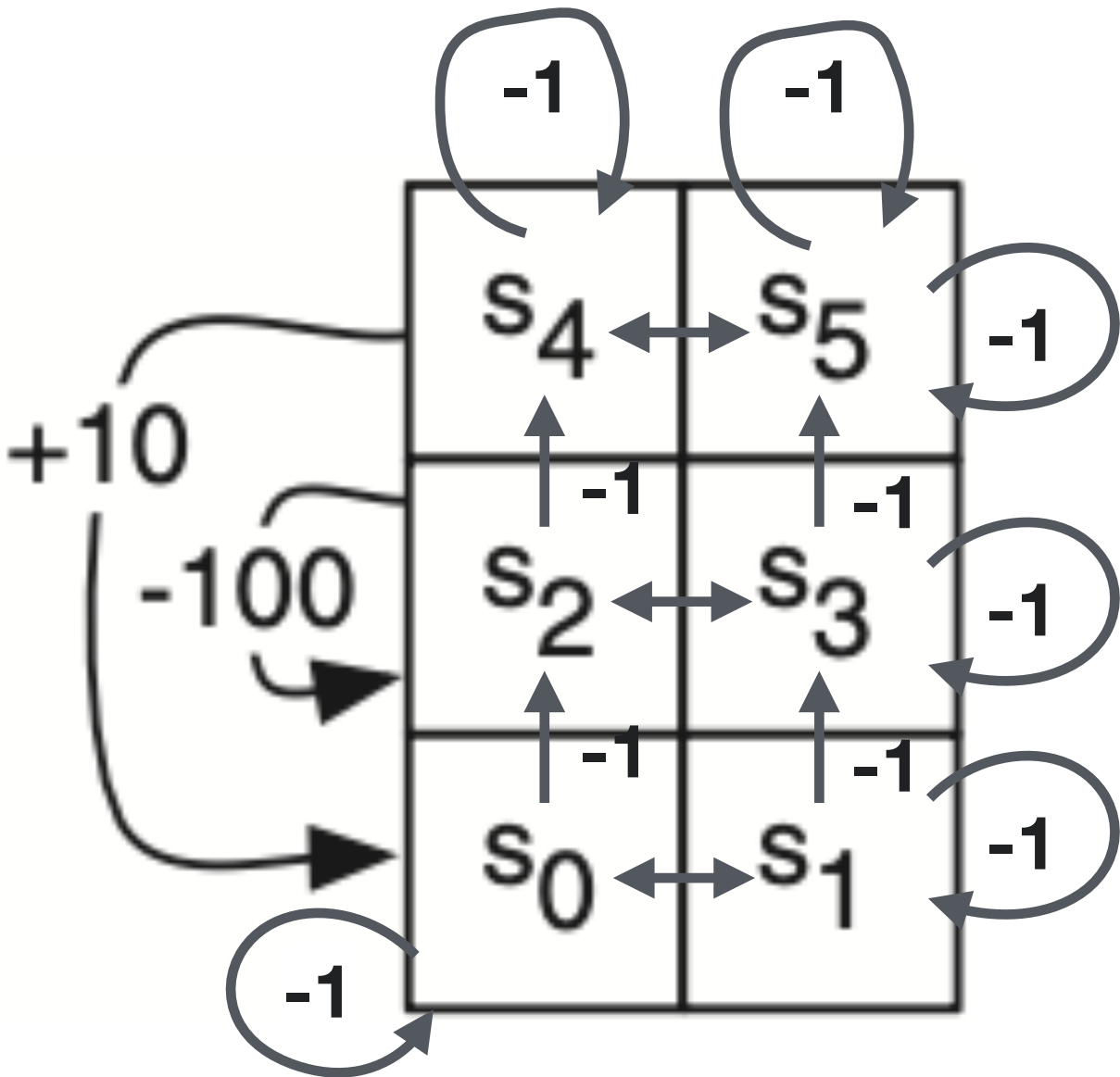
```



$$\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha[r(s, a) + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s, a)]$$

$$\alpha = 0.2, \gamma = 0.9$$

- upC** (for “up carefully”) The agent goes up, except in states s_4 and s_5 , where the agent stays still, and has a reward of -1 .
- right** The agent moves to the right in states s_0, s_2, s_4 with a reward of 0 and stays still in the other states, with a reward of -1 .
- left** The agent moves one state to the left in states s_1, s_3, s_5 . In state s_0 , it stays in s_0 and has a reward of -1 . In state s_2 , it has a reward of -100 and stays in state s_2 . In state s_4 , it gets a reward of 10 and moves to state s_0 .
- up** With a probability of 0.8 it acts like *upC*, except the reward is 0. With probability 0.1 it acts as a *left*, and with probability 0.1 it acts as *right*.



Initial Q-table

	UpC	Right	Left	Up
S0	0	0	0	0
S1	0	0	0	0
S2	0	0	0	0
S3	0	0	0	0
S4	0	0	0	0
S5	0	0	0	0

History of interaction

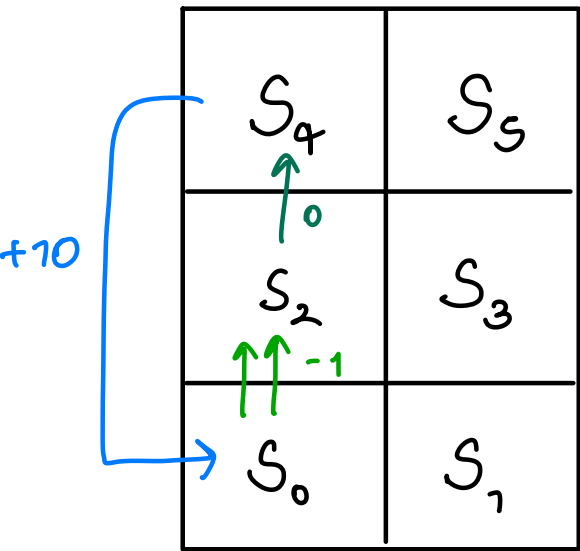
	<i>s</i>	<i>a</i>	<i>r</i>	<i>s'</i>
1	s_0	<i>upC</i>	-1	s_2
2	s_2	<i>up</i>	0	s_4
3	s_4	<i>left</i>	10	s_0
4	s_0	<i>upC</i>	-1	s_2

$$\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha[r(s, a) + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s, a)]$$

$\alpha = 0.2, \gamma = 0.9$ reward of action

History of interaction

	<i>s</i>	<i>a</i>	<i>r</i>	<i>s'</i>
1	<i>s</i> ₀	upC	-1	<i>s</i> ₂
2	<i>s</i> ₂	up	0	<i>s</i> ₄
3	<i>s</i> ₄	left	10	<i>s</i> ₀
4	<i>s</i> ₀	upC	-1	<i>s</i> ₂



1st step

	UpC	Right	Left	Up
S0	-0.2			
S1				
S2	0	0	0	0
S3				
S4				
S5				

$\hat{Q}(s_0, upC) \leftarrow 0 + 0.2[-1 + 0.9(0) - 0] = 0 + 0.2(-1)$

3rd step

	UpC	Right	Left	Up
S0	-0.2			
S1				
S2				0
S3				
S4			2	
S5				

$\hat{Q}(s_4, left) \leftarrow 0 + 0.2[10 + 0.8(0) - 0] = 0.2(10)$

2nd step

	UpC	Right	Left	Up
S0	-0.2			
S1				
S2				0
S3				
S4				
S5				

$\hat{Q}(s_2, up) \leftarrow 0 + 0.2[0 + 0.8(0) - 0] = 0$

4th step

	UpC	Right	Left	Up
S0	-0.36			
S1				
S2				0
S3				
S4			2	
S5				

$\hat{Q}(s_0, upC) = -0.2 + 0.2[-1 + 0.8(0) - (-0.2)] = -0.2 + 0.2(-0.8) = -0.2 - 0.16 = -0.36$

upC (for “up carefully”) The agent goes up, except in states s_4 and s_5 , where the agent stays still, and has a reward of -1 .

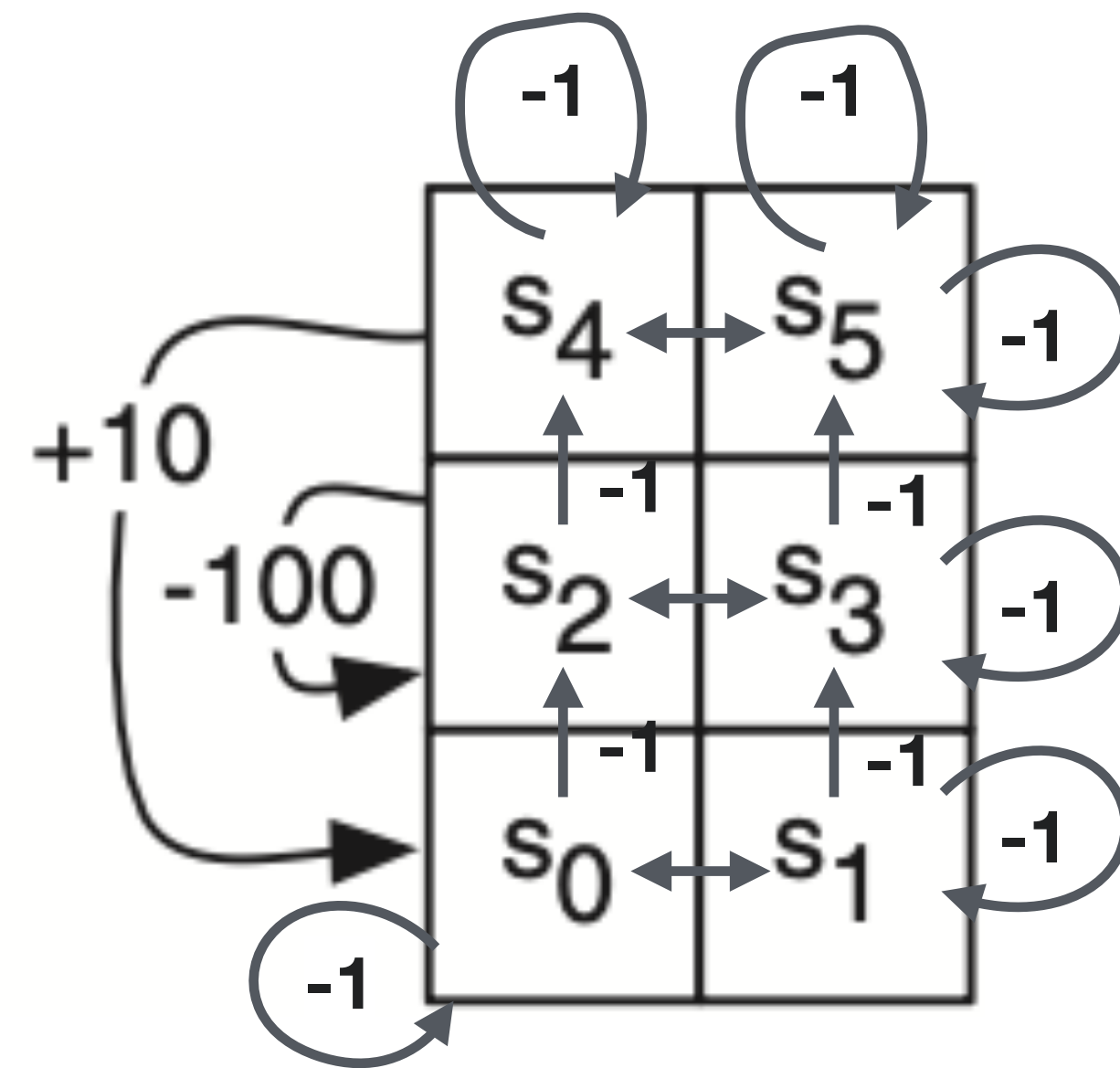
right The agent moves to the right in states s_0, s_2, s_4 with a reward of 0 and stays still in the other states, with a reward of -1 .

left The agent moves one state to the left in states s_1, s_3, s_5 . In state s_0 , it stays in state s_0 and has a reward of -1 . In state s_2 , it has a reward of -100 and stays in state s_2 . In state s_4 , it gets a reward of 10 and moves to state s_0 .

up With a probability of 0.8 it acts like *upC*, except the reward is 0. With probability 0.1 it acts as a *left*, and with probability 0.1 it acts as *right*.

	UpC	Right	Left	Up
s_0				x
s_1				x
s_2	x			
s_3				x
s_4			x	
s_5			x	

พละพละ
ดึดนุ้ยนู๋



Optimal from TD learning $\pi^*(s) = \arg \max_a Q(s, a)$

$\pi^*(s_0) = \text{up}$

$\pi^*(s_1) = \text{up}$

$\pi^*(s_2) = \text{upC}$

$\pi^*(s_3) = \text{up}$

$\pi^*(s_4) = \text{left}$

$\pi^*(s_5) = \text{left}$

เทคนิค: อดการด้วย สำหรับ Q learning
ใน Neural network

วิธีการนี้ จะให้ optimal กับ visit ทุก state

Generalizing from Examples - Deep Q Learning

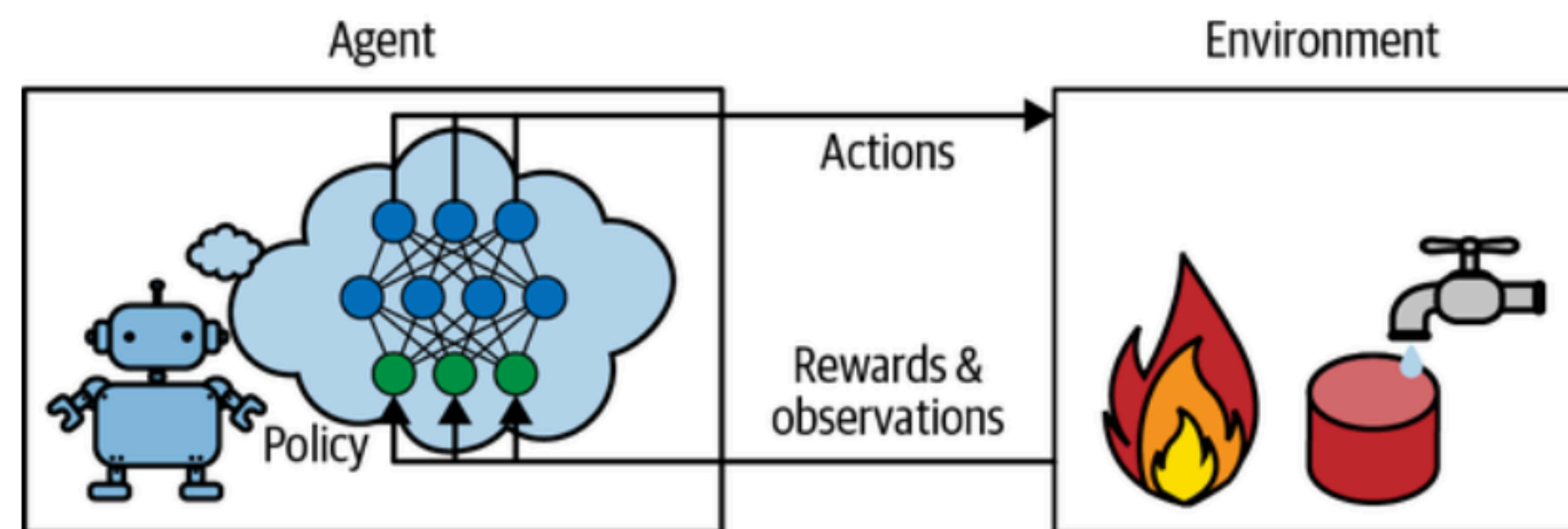
ดูที่ model ที่ใช้ Q learning จากข้อมูล train
มาหาผลลัพธ์แล้ว

Q-learning requires that all state-action pairs are to be visited infinitely often for $\hat{Q}$ to converge to actual Q .

Impossible to learn when the action-state space gets large (infinite or prohibitively large), the cost of execution actions is high.

Substitute a (deep) neural network for the lookup table and use each $\hat{Q}(s, a)$ update as a training example.

- ◆ One network with state and action as input.
- ◆ Train a separate network for each action with states as input.



$$r(s_t) + \gamma r(s_{t+1}) + \dots$$

$$V^\pi(s_t) \triangleq r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

← *discount state*

$$\hat{V}(s) \leftarrow \max_a [r(s, a) + \gamma \hat{V}(\delta(s, a))]$$

$$Q(s_t, a_t) = \max_{a_{t+1}, a_{t+2}, \dots} \left\{ r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \gamma^2 r(s_{t+2}, a_{t+2}) + \dots \right\}$$

← *discount state, action*

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

$$\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha \left[\underbrace{r(s, a) + \gamma \max_{a'} \hat{Q}(s', a')}_{\text{Actual return = Observed reward + Discounted future reward}} - \underbrace{\hat{Q}(s, a)}_{\text{Current estimated return}} \right], \quad 0 < \alpha < 1$$